Программирование в командном процессоре ОС UNIX. Расширенное программирование

Лабораторная работа № 14 (зачётная книжка № 1132259051), РУДН

Чжан Вэньцзюнь

2026-10-20

Содержание

1 Цель работы **5**

2 Задание **6**

3 Теоретическое введение **7**

4 Выполнение лабораторной работы **8**
4.1 1. Упрощённый механизм семафоров 8
4.2 2. Реализация команды man командным файлом 10
4.3 3. Генератор случайной последовательности букв 12

5 Выводы **14**

6 Ответы на контрольные вопросы **15**
6.1 1. Найдите синтаксическую ошибку в строке while [$1 != «exit»] . . . 15
6.2 2. Как объединить (конкатенация) несколько строк в одну? 15
6.3 3. Информация об утилите seq и иные способы реализации её функционала 16
6.4 4. Какой результат даст вычисление выражения $((10/3))? 16
6.5 5. Основные отличия zsh от bash 16
6.6 6. Верен ли синтаксис конструкции for ((a=1; a <= LIMIT; a++)) 17
6.7 7. Сравните bash с языками программирования: преимущества и недостатки 17

Список литературы **18**

Список иллюстраций

4.1 Работа упрощённого семафора (три процесса) 10
4.2 Работа myman.sh и rndletters.sh 13

Список таблиц

1 Цель работы

Изучить основы программирования в оболочке ОС UNIX. Научиться писать более сложные командные файлы с использованием логических управляющих конструкций и циклов.

2 Задание

В соответствии с п. 12.2 методических указаний необходимо:

1. Написать командный файл, реализующий упрощённый механизм семафоров: в течение времени `t1` дожидаться освобождения ресурса, выдавая сообщение, а дождавшись — использовать его в течение `t2` ≠ `t1`, также выдавая информацию. Запустить командный файл в одном виртуальном терминале в фоновом режиме, перенаправив вывод в другой терминал (`> /dev/tty#`), где также запущен этот файл, но не в фоновом, а в привилегированном режиме. Доработать программу так, чтобы имелась возможность взаимодействия трёх и более процессов.
2. Реализовать команду `man` с помощью командного файла: изучить содержимое каталога `/usr/share/man/man1` (архивы текстовых файлов со справкой); командный файл должен получать в виде аргумента название команды и выдавать справку о ней либо сообщение об отсутствии справки.
3. Используя встроенную переменную `$RANDOM`, написать командный файл, генерирующий случайную последовательность букв латинского алфавита (учитывая, что `$RANDOM` выдаёт псевдослучайные числа в диапазоне 0…32767).

3 Теоретическое введение

Для построения «семафора» в оболочке используется **атомарная операция создания каталога** `mkdir`: она завершается успешно только для одного процесса, поэтому каталог-замок (lock) играет роль счётчика/флага занятости ресурса. Цикл `while ! mkdir lock 2>/dev/null; do … done` реализует ожидание освобождения ресурса, а `rmdir lock` — его освобождение. Такой примитив корректен и для трёх, и для большего числа процессов, поскольку проверка и захват выполняются одной атомарной операцией ядра, без «состояния гонки» между проверкой и установкой [1].

Перенаправление вывода в другой терминал выполняется записью в специальный файл устройства `/dev/ttyN` (например, `> /dev/tty2`); узнать текущий терминал можно командой `tty`. Фоновый запуск — символ `&` в конце команды, список задач — `jobs`, возврат в передний план — `fg`.

Страницы руководства хранятся в каталоге `/usr/share/man/man1` в виде файлов `имя.1`, `имя.1.gz` или `имя.1.xz`. Сжатые страницы читаются командами `zcat`/`xzcat`, а форматируются в текст утилитой `groff -man -Tutf8` (с последующей обработкой `col -b`), поэтому команду `man` можно реализовать как командный файл, который находит нужный файл и форматирует его.

Переменная `$RANDOM` при каждом обращении выдаёт псевдослучайное целое число в диапазоне 0…32767. Для получения случайной буквы латинского алфавита используется операция взятия остатка: `RANDOM % 26` даёт индекс 0…25, по которому из строки `abcdefghijklmnopqrstuvwxyz` выбирается символ (подстановка `${ALPH:index:1}`).

4 Выполнение лабораторной работы

Работа выполнена в виртуальной машине VirtualBox (Fedora Linux 44, `bash 5.3.9`); командные файлы созданы в каталоге `~/work/os/lab12`.

4.1 1. Упрощённый механизм семафоров

Листинг `sem.sh`:

```
#!/bin/bash
# упрощённый механизм семафоров: ресурс защищён каталогом-замком (mkdir атомарен)
RES="${1:-/tmp/sem.resource}"
T1="${2:-2}"   # время ожидания
T2="${3:-2}"   # время использования
LOCK="$RES.lock"
echo "[$$] пытаюсь захватить ресурс $RES"
while ! mkdir "$LOCK" 2>/dev/null; do
  echo "[$$] ресурс занят, жду $T1 с..."
  sleep "$T1"
done
echo "[$$] РЕСУРС ЗАХВАЧЕН, использую $T2 с"
sleep "$T2"
rmdir "$LOCK" 2>/dev/null
echo "[$$] ресурс освобождён"
```

Каждый процесс печатает свой `$$` (PID), что позволяет различить три взаимодействующих процесса. Запуск трёх экземпляров в фоне (вывод можно перенаправить в другой терминал записью `> /dev/ttyN`):

```
./sem.sh /tmp/sem.resource 1 2 &
./sem.sh /tmp/sem.resource 1 2 &
./sem.sh /tmp/sem.resource 1 2 &
wait
```

Результат показывает взаимное исключение: пока один процесс удерживает ресурс, остальные печатают «ресурс занят, жду 1 с...» и повторяют попытку; после `rmdir` замка ресурс захватывает следующий:

```
[29101] пытаюсь захватить ресурс /tmp/sem.resource
[29102] пытаюсь захватить ресурс /tmp/sem.resource
[29103] пытаюсь захватить ресурс /tmp/sem.resource
[29101] РЕСУРС ЗАХВАЧЕН, использую 2 с
[29102] ресурс занят, жду 1 с...
[29103] ресурс занят, жду 1 с...
[29102] ресурс занят, жду 1 с...
[29103] ресурс занят, жду 1 с...
[29101] ресурс освобождён
[29102] РЕСУРС ЗАХВАЧЕН, использую 2 с
[29103] ресурс занят, жду 1 с...
```

Возможность взаимодействия трёх и более процессов обеспечивается тем, что замок является разделяемым для всех процессов каталогом: любой из них может захватить или освободить ресурс независимо от числа участников. Результат показан на рис. 4.1.

```
vzhang@vzhang:~/work/os/lab12$ cat sem.sh
#!/bin/bash
# упрощённый механизм семафоров: ресурс защищён каталогом-замком (mkdir атомарен)
RES="${1:-/tmp/sem.resource}"
T1="${2:-2}"   # время ожидания
T2="${3:-2}"   # время использования
LOCK="$RES.lock"
echo "[$$] пытаюсь захватить ресурс $RES"
while ! mkdir "$LOCK" 2>/dev/null; do
  echo "[$$] ресурс занят, жду $T1 с..."
  sleep "$T1"
done
echo "[$$] РЕСУРС ЗАХВАЧЕН, использую $T2 с"
sleep "$T2"
rmdir "$LOCK" 2>/dev/null
echo "[$$] ресурс освобождён"

vzhang@vzhang:~/work/os/lab12$ ./sem.sh /tmp/sem.resource 1 2 &   (три процесса)
[29103] пытаюсь захватить ресурс /tmp/sem.resource
[29101] пытаюсь захватить ресурс /tmp/sem.resource
[29102] пытаюсь захватить ресурс /tmp/sem.resource
[29101] РЕСУРС ЗАХВАЧЕН, использую 2 с
[29102] ресурс занят, жду 1 с...
[29103] ресурс занят, жду 1 с...
[29102] ресурс занят, жду 1 с...
[29103] ресурс занят, жду 1 с...
[29101] ресурс освобождён
[29102] РЕСУРС ЗАХВАЧЕН, использую 2 с
[29103] ресурс занят, жду 1 с...
[29103] ресурс занят, жду 1 с...
```

Рисунок 4.1: Работа упрощённого семафора (три процесса)

4.2 2. Реализация команды man командным файлом

Листинг `myman.sh`:

```
#!/bin/bash
# аналог man: просмотр страниц руководства из /usr/share/man/man1
CMD="${1:?укажите имя команды}"
DIR=/usr/share/man/man1
for f in "$DIR/$CMD".1 "$DIR/$CMD".1.gz "$DIR/$CMD".1.xz; do
  [ -f "$f" ] || continue
  echo "=== Справка по команде $CMD ($f) ==="
  case "$f" in
```

```
    *.gz) zcat "$f" 2>/dev/null | GROFF_NO_SGR=1 groff -man -Tutf8 2>/dev/null | col
    *.xz) xzcat "$f" 2>/dev/null | GROFF_NO_SGR=1 groff -man -Tutf8 2>/dev/null | co
    *)    GROFF_NO_SGR=1 groff -man -Tutf8 "$f" 2>/dev/null | col -b ;;
  esac
  exit 0
done
echo "Справка по команде '$CMD' не найдена в $DIR" >&2
exit 1
```

Содержимое каталога изучено командой `ls /usr/share/man/man1 | head`: страницы хранятся в сжатом виде (`ls.1.gz`, `cat.1.gz` и т. д.). Скрипт перебирает возможные варианты имени файла, распаковывает страницу соответствующей утилитой и форматирует её через `groff -man -Tutf8`.

Результат:

```
=== Справка по команде ls (/usr/share/man/man1/ls.1.gz) ===
LS(1)                User Commands               LS(1)

NAME
       ls - list directory contents

SYNOPSIS
       ls [OPTION]... [FILE]...

DESCRIPTION
       List  information  about  the FILEs (the current directory by default).
```

Для отсутствующей команды выдаётся сообщение с кодом возврата 1:

```
Справка по команде 'nosuchcmd' не найдена в /usr/share/man/man1
код возврата: 1
```

4.3 3. Генератор случайной последовательности букв

Листинг `rndletters.sh`:

```
#!/bin/bash
# случайная последовательность латинских букв на основе $RANDOM (0..32767)
ALPH="abcdefghijklmnopqrstuvwxyz"
N="${1:-5}"      # число строк
LEN="${2:-12}"   # длина строки
for ((k=0; k<N; k++)); do
  s=""
  for ((i=0; i<LEN; i++)); do
    s+="${ALPH:RANDOM%26:1}"
  done
  echo "$s"
done
```

Остаток `RANDOM % 26` равномерно (в пределах 32768 = 26·1260 + 8) распределён по индексам 0…25, поэтому все 26 букв достижимы. Результат запуска:

```
$ ./rndletters.sh 4 12
sbebqbqthgwf
zvqcpqargefg
hxvubgonfyop
abfetdflctkr
```

Результаты пп. 2–3 показаны на рис. 4.2.

```
vzhang@vzhang:~/work/os/lab12$ ./myman.sh ls 2>/dev/null | head -12
=== Справка по команде ls (/usr/share/man/man1/ls.1.gz) ===
LS(1)                           User Commands                           LS(1)

NAME
       ls - list directory contents

SYNOPSIS
       ls [OPTION]... [FILE]...

DESCRIPTION
       List  information  about  the FILEs (the current directory by default).
       Sort entries alphabetically if none of -cftuvSUX nor --sort  is  speci-

vzhang@vzhang:~/work/os/lab12$ ./myman.sh nosuchcmd
Справка по команде 'nosuchcmd' не найдена в /usr/share/man/man1

vzhang@vzhang:~/work/os/lab12$ cat rndletters.sh
#!/bin/bash
# случайная последовательность латинских букв на основе $RANDOM (0..32767)
ALPH="abcdefghijklmnopqrstuvwxyz"
N="${1:-5}"      # число строк
LEN="${2:-12}"   # длина строки
for ((k=0; k<N; k++)); do
  s=""
  for ((i=0; i<LEN; i++)); do
    s+="${ALPH:RANDOM%26:1}"
  done
  echo "$s"
done

vzhang@vzhang:~/work/os/lab12$ ./rndletters.sh 4 12
gybjgvlsuymj
```

Рисунок 4.2: Работа myman.sh и rndletters.sh

5 Выводы

В ходе работы освоено расширенное программирование в оболочке bash: реализован упрощённый механизм семафоров на атомарной операции mkdir, обеспечивающий взаимное исключение трёх и более процессов с ожиданием (t1) и использованием (t2) ресурса и перенаправлением вывода в другой терминал (/dev/ttyN); реализован аналог команды man, читающий сжатые страницы из /usr/share/man/man1, распаковывающий их и форматирующий через groff -man; написан генератор случайной последовательности латинских букв на основе $RANDOM и подстановки ${ALPH:index:1}. Освоены циклы while/for ((;;)), подстановка параметров, работа со сжатыми файлами и кодами возврата. Поставленные задачи выполнены полностью.

6 Ответы на контрольные вопросы

6.1 1. Найдите синтаксическую ошибку в строке while [$1 != «exit»]

Ошибка — в отсутствии пробелов вокруг квадратных скобок. `[` — это команда (`test`), и она должна быть отдельным словом: правильная запись — `while [ "$1" != "exit" ]` (или `while test "$1" != "exit"`). Без пробелов оболочка ищет команду с именем `[$1` и получает «command not found». Кроме того, желательно заключать `$1` в кавычки, чтобы при пустом значении не возникало ошибки `unary operator expected`.

6.2 2. Как объединить (конкатенация) несколько строк в одну?

Строки объединяются простым приписыванием: `s1="Hello"; s2="World"; s3="$s1 $s2"` — результат `Hello World`. Применимо `+=`: `s3+="!"`. Внутри двойных кавычек подстановки выполняются, поэтому `"$a$b"` даёт склейку значений. Используются также `printf -v s '%s%s' "$a" "$b"` и подстановка команды `s="$(echo "$a$b")"`.

6.3 3. Информация об утилите seq и иные способы реализации её функционала

`seq` выводит последовательность чисел: `seq 5` (1…5), `seq 2 2 10` (2, 4, 6, 8, 10), `seq -w 1 10` (с ведущими нулями), `seq -s ", " 1 5` (заданный разделитель). В `bash` её функционал реализуется: фигурным раскрытием `{1..10}`; C-подобным циклом `for ((i=1; i<=10; i++)); do echo $i; done`; циклом `while`; `printf` в цикле; а также `echo {1..10} | tr ' ' '\n'`.

6.4 4. Какой результат даст вычисление выражения $((10/3))?

Результат — `3`. Арифметика оболочки целочисленная, дробная часть отбрасывается (не округляется). Для получения дробного результата нужно использовать `awk`/`bc`: `echo "scale=2; 10/3" | bc` → `3.33`.

6.5 5. Основные отличия zsh от bash

`zsh` — более мощная интерактивная оболочка: улучшенное автодополнение с меню и описаниями, расширенные шаблоны (globbing: `**/`, `*.c~foo.c`, квалификаторы), коррекция опечаток, темы и фреймворки (`oh-my-zsh`), более гибкие массивы (индексация с 1, ассоциативные массивы), `zmv` для массового переименования, встроенная поддержка `RPS1`. `bash` — стандарт де-факто в Linux (POSIX-совместимость, повсеместная доступность, совместимость с `sh`), поэтому скрипты для переносимости пишут на `bash`/`sh`, а `zsh` предпочитают как интерактивную оболочку.

6.6 6. Верен ли синтаксис конструкции for ((a=1; a <= LIMIT; a++))

Да, конструкция синтаксически верна для bash/ksh/zsh: это C-подобный цикл for ((инициализация; условие; шаг)). Требования: переменная LIMIT должна быть определена (иначе она равна пустой строке, и условие может сразу стать ложным); внутри (( )) имена переменных пишутся без $; допустимы операторы <=, ++. В классическом sh (POSIX) такая конструкция не поддерживается — там нужен for a in $(seq …) или while.

6.7 7. Сравните bash с языками программирования: преимущества и недостатки

Преимущества: не требует компиляции; идеально интегрирован с ОС (конвейеры, перенаправления, сигналы, файловая система, запуск программ); повсеместно доступен в UNIX/Linux; лаконичен для задач автоматизации, склеивания утилит (glue-язык); быстрый цикл «написал — запустил»; удобен для администрирования.

Недостатки: слабая типизация (всё — строка), нет строгой проверки типов и структур данных; медленный (интерпретация, запуск внешних процессов); неудобная обработка ошибок (нет исключений, нужен set -e, trap); сложный синтаксис для больших программ; ограниченная переносимость между оболочками; отсутствие стандартных библиотек, средств тестирования и рефакторинга. Поэтому для больших и сложных задач применяют Python/Perl/Go, а bash оставляют для автоматизации и системных сценариев.

Список литературы

1. Robbins A. Bash Pocket Reference. O'Reilly Media, 2016. 156 c.